ARCA RH

Definición arquitectónica y plan de ejecución

Kata arquitectónico · Tercer desafío · 27 de octubre de 2026

Documento de decisión de inicio para el equipo de Tecnología de ARCA LTDA. Alcance funcional, límites de microservicios, propiedad de datos, APIs, seguridad, backlog, entregables y presentación al Directorio.

ESTADO	INTERPRETACIÓN
OBLIGATORIO	Requisito expresamente indicado en el enunciado del ingeniero.
PROPUESTO	Decisión técnica del equipo para poder comenzar, modificable mediante acuerdo documentado.
POR CONFIRMAR	El documento no proporciona una regla suficiente. No fingir que está decidida.
NO IMPLEMENTADO	Este documento fija el diseño; no acredita código, pruebas ni despliegue realizados.

Alcance temporal: arranque el 27/10/2026; cortes tentativos el 03/11 y 10/11 según exposición oral; entrega final oficial el 17/11/2026.

Fuentes principales: enunciado oficial KATA ARQUITECTÓNICO – TERCER DESAFÍO (2 páginas); explicación del docente en clase; prácticas de documentación de los desafíos previos.

1. Resumen ejecutivo y trazabilidad

ARCA LTDA. adquirió un sistema Core y descartó su módulo de gestión humana por costo. El reto es construir una solución propia, demostrable al Directorio en 2–3 sprints. El enunciado exige microservicios, base de datos relacional, una capa de presentación con impacto y documentación equivalente a desafíos anteriores.

ID	OBLIGATORIO SEGÚN KATA	PRUEBA DE ACEPTACIÓN
R-01	Gestión de personal: alta, modificación de área/cargo/remuneración y baja. Ponderación señalada: 60%.	Crear, modificar y dar de baja lógica a un funcionario.
R-02	Vacaciones: funcionario ratificado con un año de antigüedad; 15 días por gestión completada. 70%.	Mostrar elegibilidad, disponibilidad, solicitud y aprobación.
R-03	Contratos: generar contrato básico parametrizable, con fecha de ingreso, salario y tiempo de prueba. 80%.	Editar plantilla y producir PDF con datos y versión.
R-04	Boletas: generar y almacenar información de pagos. 100%.	Registrar un pago, emitir boleta y consultarla después.
R-05	Arquitectura de microservicios y datos en base relacional.	Procesos aislados, comunicación API, persistencia privada.
R-06	Presentación convincente para Directorio y documentación formal.	Demostración guiada con datos consistentes y documentos.

Nota: 60/70/80/100 % se transcriben como ponderaciones del cuadro; no se presupone si son porcentajes acumulativos, niveles o notas máximas. Todos los módulos se planifican.

2. Decisiones del equipo tomadas como propuesta

ADR	DECISIÓN	RAZÓN Y LÍMITE
ADR-01	Cinco microservicios desplegables: Identidad, Personal, Vacaciones, Contratos y Pagos.	Cuatro dominios del kata y un habilitador de acceso; no fragmentar más.
ADR-02	React + TypeScript, NestJS + TypeScript, Prisma, PostgreSQL.	Reutilizar conocimientos del grupo; minimizar curva de aprendizaje.
ADR-03	API Gateway HTTP y APIs REST síncronas internas; sin broker inicial.	Simplificar integración en tres sprints. Añadir mensajería sólo ante necesidad validada.
ADR-04	Una instancia PostgreSQL con cinco bases lógicas y usuarios distintos.	Cumple base relacional y prohíbe tablas compartidas entre servicios.
ADR-05	Docker Compose en un entorno de demostración.	Reproducibilidad local; no confundir con alta disponibilidad productiva.
ADR-06	Baja lógica de funcionarios y documentos históricos inmutables tras emisión.	Conservar trazabilidad administrativa y financiera.
ADR-07	Sin integración funcional Core en MVP; frontera de API documentada.	El kata no aporta contrato ni credenciales del Core.
ADR-08	Sin cálculo legal automático de nómina ni transferencia bancaria.	El enunciado exige generar y registrar pagos, no liquidación legal ni banca.

3. Contexto y alcance del MVP

- Actores: Administrador RR. HH.; Operador RR. HH.; Aprobador de vacaciones; Finanzas/Nómina; Directorio (consulta y visión ejecutiva).
- Dentro de alcance: los cuatro dominios del kata, autenticación, permisos, historial básico, generación de PDF, consultas filtradas, indicadores obtenidos de datos reales de la demo.

- Fuera de alcance inicial: comunicación real con el Core, pago bancario, liquidación fiscal automatizada, firma electrónica, integración con biométricos, autoservicio de empleados y aplicaciones móviles.
- Diseño extensible: la integración futura con Core se haría mediante un adaptador/versionado externo, nunca permitiendo a Core escribir directamente en las tablas de los servicios.

4. Arquitectura de contenedores (C4, nivel 2)

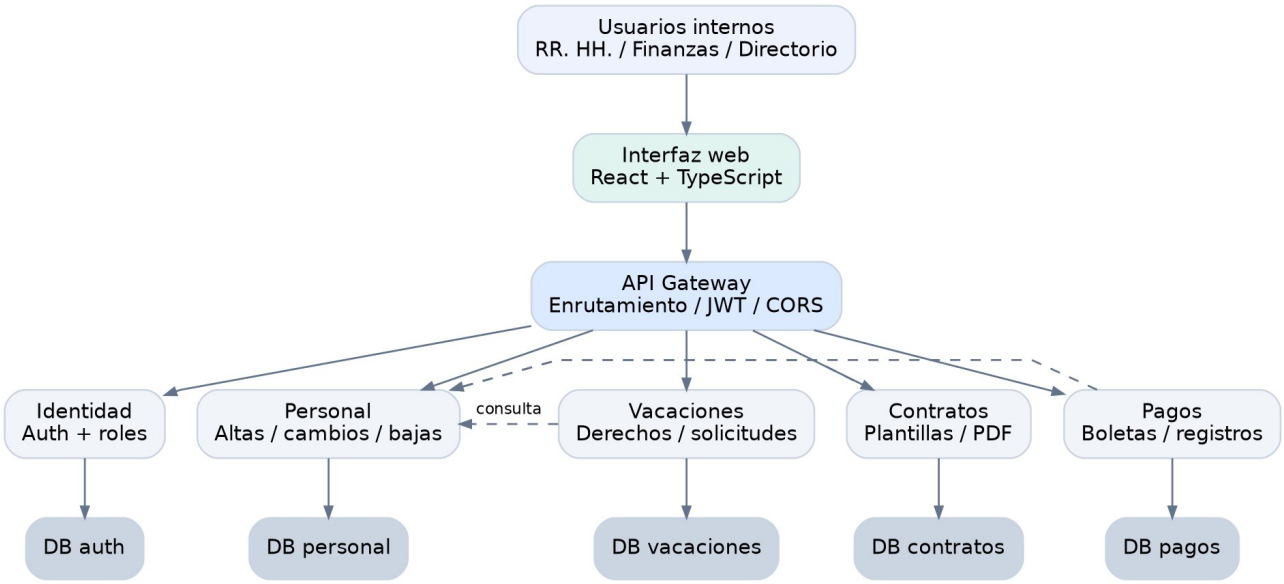


Figura 1. Vista de contenedores propuesta. Las líneas discontinuas representan consultas internas a Personal; cada servicio es dueño exclusivo de su base. No hay clave foránea entre bases de servicios.

Servicios desplegables separados: cada backend tiene su proceso/imagen, puerto privado, migraciones y credenciales. El frontend sólo consume el Gateway. El Gateway no contiene reglas de RR. HH.; autentica, limita tráfico y enruta.

5. Responsabilidades y datos por servicio

SERVICIO	RESPONSABILIDAD	TABLAS PRINCIPALES (PROPUESTAS)
Identidad	Inicio de sesión, usuarios, roles, autorización de funciones.	users, roles, user_roles, login_audit
Personal	Fuente de verdad del empleado, área, cargo, remuneración y vigencia.	employees, departments, positions, salary_history, employment_events
Vacaciones	Elegibilidad, derechos por gestión, solicitudes, aprobación y saldo.	leave_entitlements, leave_requests, leave_movements
Contratos	Plantillas versiones, datos congelados de emisión, documentos PDF.	contract_templates, contracts, contract_documents
Pagos	Registro monetario, partidas, emisión de boleta e historial.	payment_records, payment_lines, pay_slips

Campos transversales: id UUID, created_at, updated_at y actor_id donde corresponde. Los servicios externos almacenan employee_id UUID y el nombre/salario del funcionario como snapshot del momento, NO como referencia SQL entre bases.

5.1 Datos mínimos y restricciones

- Personal: employee_code único; documento único si existe; nombres; fecha_ingreso; ratificado_en nullable; departamento_id; cargo_id; salario_actual DECIMAL(14,2); moneda; estado ACTIVO/BAJA; fecha_baja; historial salarial con fecha de vigencia.

- Vacaciones: entitlement(employee_id, gestion, dias_otorgados=15, dias_usados); request(employee_id, fecha_inicio, fecha_fin, dias_solicitados, estado PENDIENTE/APROBADA/RECHAZADA/CANCELADA, aprobador_id).
- Contratos: template(id, version, cuerpo/variables_permitidas, activo); contract(employee_id, template_version, fecha_ingreso, salario_snapshot, periodo_prueba_dias, estado BORRADOR/EMITIDO, emitido_en); pdf privado ligado a contrato.
- Pagos: payment(employee_id, periodo, moneda, salario_base_snapshot, otros_abonos, descuentos, neto, estado BORRADOR/EMITIDO/PAGADO, fecha_pago_nullable); partidas detalladas y boleta versionada.
- Identidad: usuario, hash seguro de contraseña, estado, roles; vinculación opcional a empleado sólo por employee_id. No almacenar contraseñas en texto ni mostrarlas en logs.

Base de datos: PostgreSQL físico único, con db_identity, db_personal, db_vacations, db_contracts, db_payments y un usuario SQL distinto por base. Se bloquean consultas cruzadas y se prohíbe Prisma accediendo a base ajena.

6. Reglas de negocio y flujos

6.1 Personal

1. Alta: crear funcionario con código único, área, cargo, fecha de ingreso, remuneración y estado ACTIVO; registrar autor y fecha.
2. Modificación: cambiar área/cargo/remuneración; conservar historial de salario y de movimientos, sin borrar información anterior.
3. Baja: registrar fecha y motivo, pasar a BAJA y deshabilitar acciones nuevas pertinentes. No eliminar históricamente contratos ni pagos.

6.2 Vacaciones

1. Elegibilidad: empleado ratificado (ratificado_en no nulo), con al menos un aniversario completo de fecha_ingreso, con estado laboral válido al solicitar.
2. Derecho: 15 días por gestión de antigüedad completada, identificados por cada aniversario; el cálculo de días hábiles/calendario y el arrastre quedan por confirmar con el docente.
3. Solicitud: elegir empleado y fechas, indicar cantidad de días sujeta a la política de conteo; bloquear solicitudes con saldo insuficiente o empleado no elegible.
4. Aprobación: transacción atómica que comprueba saldo disponible y marca APROBADA una sola vez; rechazo no consume días; el saldo no puede volverse negativo.

Decisión conservadora: registrar cada gestión con su propio movimiento para NO perder días al cambiar de año. No inventar reglas de caducidad, acumulación legal o días feriados; quedarán parametrizadas/pendientes.

6.3 Contratos

1. Seleccionar empleado existente; consultar datos actuales vía API de Personal; elegir plantilla activa.
2. Previsualizar ingreso, salario y periodo de prueba. Validar variables permitidas y guardar versión y snapshot utilizados.
3. Generar PDF no editable desde la aplicación; guardar metadatos y almacenamiento privado. Los contratos emitidos no cambian si más tarde cambia el sueldo.

6.4 Boletas

1. Finanzas elige empleado y periodo, consulta salario vigente según periodo vía Personal y registra importes de abonos/descuentos; validar DECIMAL y neto >= 0.
2. Antes de emitir, revisar valores; al emitir, conservar snapshot y crear PDF descargable por usuarios autorizados.

3. Separar EMITIDA de PAGADA. Sólo marcar PAGADA si existe fecha y referencia/evidencia correspondiente; la emisión de PDF no transfiere dinero.

La deducción impositiva, aportes y cálculo salarial legal NO son funcionalidad inferible del enunciado. Si el docente la exige, convertirla en nuevo requerimiento validado y estimado.

7. Contratos HTTP de la API (borrador v1)

MÉTODO / RUTA EXTERNA	OBJETIVO Y AUTORIZACIÓN
POST /api/v1/auth/login	Token de acceso; usuario activo.
GET /api/v1/auth/me	Perfil/roles del usuario autenticado.
POST /api/v1/employees	Alta personal; RR. HH.
GET /api/v1/employees	Listado paginado con filtros por área/estado; permisos.
PATCH /api/v1/employees/{id}	Modificar área, cargo y salario; RR. HH.
POST /api/v1/employees/{id}/termination	Baja lógica auditada; RR. HH.
GET /api/v1/vacations/balance/{employeeId}	Saldo y elegibilidad; RR. HH./aprobador.
POST /api/v1/vacations/requests	Crear solicitud de vacaciones; RR. HH.
POST /api/v1/vacations/requests/{id}/approve	Aprobar atómicamente; aprobador.
POST /api/v1/vacations/requests/{id}/reject	Rechazar y registrar motivo; aprobador.
POST /api/v1/contracts	Generar borrador desde plantilla; RR. HH.
POST /api/v1/contracts/{id}/issue	Emitir y congelar documento.
GET /api/v1/contracts/{id}/pdf	Descarga autenticada y autorizada.
POST /api/v1/payments	Crear registro de pago/borrador; Finanzas.
POST /api/v1/payments/{id}/issue	Emitir boleta y congelar datos; Finanzas.
GET /api/v1/payments/{id}/pdf	Descargar boleta; Finanzas y roles autorizados.

Convención de API: OpenAPI/Swagger por servicio; DTO validados; fechas ISO 8601; importes como strings decimales JSON; error uniforme {code, message, requestId}; versionado /v1; IDs UUID; paginación; 401 sin sesión, 403 sin permiso, 404 recurso inexistente, 409 conflicto, 422 validación de dominio.

Comunicación interna: Vacaciones→Personal GET /internal/employees/{id}/eligibility; Contratos→Personal GET /internal/employees/{id}/contract-data; Pagos→Personal GET /internal/employees/{id}/compensation?period=AAAA-MM. Se autentican llamadas internas, se aplica timeout y se propagan requestId.

8. Seguridad, confiabilidad y desempeño

- Autenticación: contraseñas con Argon2id; JWT firmado con secreto protegido fuera del repositorio; caducidad y rechazo de usuarios inactivos. Sin cuentas de prueba con credenciales publicadas.
- Autorización RBAC en Gateway Y en microservicio; revisar objeto individual y campos sensibles (salario, contrato, boleta), no sólo el menú React.
- Transporte HTTPS en exposición pública; red Docker privada entre servicios; no publicar puertos PostgreSQL ni servicios internos en internet.
- Auditoría: registro de altas, bajas, cambios de remuneración, aprobaciones, emisión de contratos y boletas, con usuario, fecha e identificador correlacionado.
- Integridad: transacciones dentro de cada servicio; snapshots inmutables; evitar transacciones distribuidas; si Personal está caído, suspender emisiones que requieran validar empleado, no improvisar datos.
- Operación: healthchecks, logs estructurados sin datos personales, pruebas de backup/restauración de DB de demostración; timeout razonable e indicadores de error.
- Datos de demo: exclusivamente sintéticos; no usar datos personales reales de compañeros o funcionarios. Ocultar remuneraciones individuales al rol Directorio salvo autorización específica.

No se afirma haber certificado ISO, cumplimiento jurídico, alta disponibilidad o cifrado de reposo: son posibles pasos posteriores, no trabajos realizados.

9. Presentación para el Directorio: pantallas y experiencia

PANTALLA	QUÉ DEBE MOSTRAR
Acceso	Inicio de sesión claro, identidad visual ARCA RH, navegación por permisos.
Dashboard	Total de personal activo, vacaciones pendientes, contratos emitidos y boletas del periodo. Indicadores reales; nunca métricas fingidas.
Personal	Tabla con búsqueda, filtros, alta guiada, ficha del funcionario e historial de cambios.
Vacaciones	Saldo visible, elegibilidad con explicación, calendario/fechas y flujo pendiente→aprobada.
Contratos	Plantilla editable con variables permitidas, vista previa y botón PDF; historial por trabajador.
Boletas	Periodo, componentes del pago, estado y PDF imprimible; historial y filtros.
Estados UX	Loading, vacíos, errores de autorización/servicio, confirmaciones y validaciones legibles.

Estilo: moderno y sobrio, tipografía legible, colores institucionales elegidos por el equipo, diseño responsivo, consistencia de componentes, sin animaciones distractoras. La UX “de impacto” se demuestra con flujo completo, no sólo estética.

10. Plan de sprints y dependencias

FECHA	OBJETIVO	EVIDENCIA AL CIERRE
27 oct - 03 nov / Sprint 1 (propuesto)	Infraestructura y contratos; identidad; Personal funcional; esqueletos separados de los cinco servicios; base React.	docker compose levanta; login; alta/modificación/baja; migraciones; Swagger; PR + pruebas de Personal.
04 - 10 nov / Sprint 2 (propuesto)	Vacaciones completas; Contratos parametrizables; base de Pagos; interfaz integrada; primeras pruebas cruzadas.	Elegibilidad y saldo; aprobar/rechazar; contrato PDF; registro de pago en borrador; evidencias.
11 - 17 nov / Sprint 3 (final oficial)	Pagos/boletas completas; endurecimiento de seguridad; documentación C4; datos sintéticos; demo y correcciones.	Boleta PDF persistente; suite E2E; diagramas; instrucciones de arranque; presentación al Directorio.

El enunciado sólo fija de modo inequívoco el cierre del 17 de noviembre; 03/11 y 10/11 son hitos recibidos en la explicación oral y aquí se usan para planificar. Si el docente mueve el primer corte, replanificar capacidad sin cambiar la fecha final sin autorización.

11. Distribución de trabajo para la reunión del 27/10

FRENTE / ROL	ACTIVIDADES HOY Y ESTA SEMANA
A — Arquitectura / Integración	Aprobar ADRs, diagramas C4, monorepo, contratos /v1, Gateway, Compose y CI; coordinar las integraciones.
B — Personal	Modelo empleado/sueldo, migración, CRUD, baja lógica, endpoints internos y pruebas.
C — Vacaciones	Reglas elegibilidad, modelos, transacciones de aprobación y casos de borde.
D — Contratos	Modelo plantillas, snapshots, PDF, variables permitidas y previsualización.
E — Pagos / UI	Modelo pagos, emisión de boletas, y/o integración frontend y Dashboard; priorizar coordinación con Personal.

Son frentes, no nombres de personas. Si hay menos de cinco integrantes, unir A con apoyo frontend y asignar una función adicional por persona; cada historia debe tener un responsable único y revisor distinto cuando sea posible.

12. Backlog inicial listo para registrar en Jira

EPIC	HISTORIAS (HU) / CRITERIO MÍNIMO
EP-00 Arquitectura	HU-00 repositorio/Compose/DB; HU-01 Gateway y JWT; HU-02 componentes UI y dashboard inicial.
EP-01 Personal	HU-03 alta; HU-04 modificar área/cargo/salario con historial; HU-05 baja lógica; HU-06 listado y filtros.
EP-02 Vacaciones	HU-07 consulta elegibilidad y 15 días; HU-08 solicitud; HU-09 aprobar/rechazar con control de saldo.
EP-03 Contratos	HU-10 plantillas variables y versiones; HU-11 contrato borrador; HU-12 PDF emitido e historial.
EP-04 Pagos	HU-13 borrador/partidas/neto; HU-14 emitir PDF; HU-15 historial y estado de pago.
EP-05 Calidad	HU-16 RBAC/auditoría; HU-17 casos E2E; HU-18 documentación C4 y demo final.

Convención recomendada: Epic → Historia → Tareas/Subtareas. Cada HU debe documentar actor, objetivo, precondiciones, criterios de aceptación verificables, dueño, dependencias y evidencia. No inventar story points antes de que el equipo estime.

13. Pruebas funcionales mínimas de aceptación

CASO	ENTRADA Y RESULTADO ESPERADO
T01	Alta de funcionario válido: creado, visible en listado, código único.
T02	Código repetido: 409; no hay duplicado.
T03	Cambio de salario: se actualiza vigente y se conserva historial.
T04	Baja de personal: no desaparecen contratos/boletas históricas.
T05	No ratificado: vacaciones denegadas aunque cumpla antigüedad.
T06	Ratificado con <1 año: sin derecho a 15 días.
T07	Ratificado con una gestión completada: recibe derecho de 15 días.
T08	Dos aprobaciones que excederían saldo: no permitir saldo negativo.
T09	Contrato emitido conserva sueldo original después de modificación.
T10	Contrato parametrizable produce PDF legible con los campos obligatorios.
T11	Boleta se emite y recupera con importes coherentes y mismo snapshot.
T12	Emitir PDF NO marca transacción financiera como PAGADA.
T13	Usuario sin rol intenta ver boleta ajena: 403 aun con URL directa.
T14	Personal inalcanzable: no emitir documentos que requieren nueva validación.
T15	Al reiniciar contenedores, datos siguen en almacenamiento persistente.

14. Demostración final ante Directorio

1. Iniciar sesión como Administrador RR. HH.; abrir dashboard con contadores derivados de registros reales.
2. Crear un funcionario ficticio, asignar área/cargo/salario y visualizar su ficha.
3. Modificar remuneración y exhibir el historial; explicar por qué los documentos ya emitidos conservarán snapshot.
4. Mostrar empleado ratificado con un año cumplido; solicitar cinco días de vacaciones y aprobar, verificando saldo. Mostrar rechazo de solicitud excesiva.
5. Crear contrato desde plantilla parametrizable y descargar PDF.
6. Como Finanzas, registrar abonos/descuentos, emitir boleta PDF y consultarla por periodo; diferenciar EMITIDA de PAGADA.
7. Mostrar que un rol sin permiso no visualiza salarios/boletas, luego explicar C4, microservicios independientes y propiedad de datos.

Reservar datos de demostración sintéticos para ambos casos: un empleado elegible y otro no elegible. Mantener plan de respaldo de demo con capturas/exportaciones, sin presentarlo como sustituto de software ejecutándose.

15. Riesgos, preguntas abiertas y decisiones que NO debemos inventar

TEMA	RIESGO / MEDIDA / ESTADO
Integración Core	No hay API, eventos, SSO ni especificaciones del Core. Fuera de MVP operativo; incluir contrato futuro y preguntar al docente.
Vacaciones	“Ratificado”, gestión, días hábiles/calendario, feriados y arrastre no están desarrollados. Registrar preguntas y parametrizar; no vender suposiciones como política legal.
Remuneración	El kata no especifica moneda, bonos ni descuentos obligatorios. Mantener moneda por registro; aprobar casos de ejemplo.
Boletas	No se exige calcular planilla fiscal ni operar banco. Confirmar alcance antes de ampliar.
Contratos	No se indica plantilla legal, firma ni obligatoriedad de anexos. Crear formato parametrizable neutro para demo.
Esfuerzo	2–3 sprints hacen riesgosa la fragmentación excesiva. Limitar a 4 dominios + identidad; priorizar flujo integral.
Microservicios	Un solo repositorio NO implica monolito si son procesos, APIs, despliegues y datos separados. Demostrarlo técnicamente.
Porcentaje de costo	El “tercio” es dato relativo, no monto. Sin cotización del Core, costos de infraestructura, licencias y horas, no afirmar ahorro numérico.

16. Lista de cierre de la reunión: 27/10/2026

- ☐ Ratificar ADR-01 a ADR-08 como baseline de equipo (documentar cambios).
- ☐ Crear repositorio y carpetas separadas de servicios, frontend, infraestructura, docs y tests.
- ☐ Crear tablero Jira con 6 epics, HU-00 a HU-18 y dueños reales (sin puntos inventados).
- ☐ Definir nombres de endpoints y contratos OpenAPI v1; acordar UUID y convenciones de errores.
- ☐ Preparar Docker Compose, PostgreSQL con 5 bases separadas y credenciales locales por servicio.
- ☐ Crear scaffolds NestJS y React; definir rama principal y revisión PR.
- ☐ Acordar dos datos sintéticos de prueba, uno elegible para vacaciones y otro no.
- ☐ Registrar formalmente las preguntas al docente; no bloquear el esquema mínimo.
- ☐ Confirmar responsables, demo de Sprint 1 y Definition of Done de cada historia.

Resultado HONESTO al 27/10: arquitectura y backlog propuestos/documentados en este artefacto. No se afirma que el repositorio, contenedores, pantallas, endpoints o pruebas estén construidos.

17. Estructura de repositorio propuesta

```
arca-rh/
  apps/
    web/
    api-gateway/
    identity-service/
    personnel-service/
    vacation-service/
    contract-service/
    payment-service/
  packages/
    api-contracts/        # DTO/OpenAPI; NO modelos Prisma compartidos
  infra/
    docker-compose.yml
    postgres-init/
  docs/
    adr/  c4/  api/  business-rules/  sprint-evidence/
  tests/
    e2e/
```

El monorepo comparte tooling y contratos, pero no entidades ORM ni accesos SQL entre servicios. Prisma y migraciones permanecen dentro de cada servicio. Se podrán construir/ejecutar y reiniciar por separado.

18. Fuentes y criterios profesionales

- KATA ARQUITECTÓNICO – TERCER DESAFÍO, ARCA LTDA., 27/10/2026, págs. 1–2. Fuente obligatoria del desafío.
- Explicación oral del ingeniero el 27/10/2026, según transcripción compartida por el equipo. Fuente contextual para hitos 03/11 y 10/11.
- Microsoft Azure Architecture Center, Microservices Architecture: https://learn.microsoft.com/en-us/azure/architecture/microservices/
- Microsoft, Data Considerations for Microservices: https://learn.microsoft.com/en-us/azure/architecture/microservices/design/data-considerations
- NestJS, Microservices Basics: https://docs.nestjs.com/microservices/basics
- Docker Compose, Startup Order: https://docs.docker.com/compose/how-tos/startup-order/
- OWASP API Security Top 10 – 2023: https://api-security.owasp.org/editions/2023/en/0x11-t10/
- Prisma, Database Models and Relational PostgreSQL documentation: https://www.prisma.io/docs/

ADVERTENCIA DE ALCANCE. Este documento es un diseño de inicio suficientemente concreto para distribuir desarrollo; no constituye diagnóstico de arquitectura real de ARCA, prueba de rendimiento, auditoría legal ni conformidad normativa.